



INSTITUT TEKNOLOGI SUMATERA
Program Studi Teknik Informatika

TUGAS BESAR MANAJEMEN BASIS DATA

**Penerapan Access Control dan Pencegahan SQL Injection pada
Basis Data Rekam Medis Elektronik**

Mata Kuliah: IF25-40405 / Manajemen Basis Data
Semester Genap 2025/2026

Dosen Pengampu:

1. Meida Cahyo Untoro, S.Kom., M.Kom.
2. Raidah Hanifah, S.T., M.T.

Kelompok	Kelompok 7
Bidang Topik	Keamanan, Keandalan & Etika Data
Anggota Kelompok	1. Margaretta Angela Manulang - 123140010 2. Ade Putri Tifani - 123140011 3. Natasya Felisita Br Ginting - 123140017 4. Jesika Filosovi Br P-A - 123140044 5. Nabila Ramadhani M. - 123140062 6. Annisa Salsabila - 123140070 7. Willy Syifa Luthfia -123140071 8. Silvia - 123140133
Tanggal Pengumpulan	29 / 05 / 2026

Abstrak

Transformasi digital di sektor kesehatan mendorong penggunaan Rekam Medis Elektronik (RME) secara luas untuk meningkatkan efisiensi layanan dan integrasi data pasien. Namun, penyimpanan data medis yang bersifat sensitif juga meningkatkan risiko ancaman keamanan, terutama serangan SQL Injection dan penyalahgunaan hak akses pengguna. Penelitian ini merancang mekanisme keamanan basis data RME melalui integrasi Role-Based Access Control (RBAC) dan deteksi SQL Injection pada level basis data.

Sistem dirancang menggunakan pendekatan *layered architecture* dengan *security middleware* berbasis Python yang berfungsi melakukan validasi hak akses pengguna dan mendeteksi pola query berbahaya sebelum diteruskan ke database MySQL. RBAC diimplementasikan melalui mekanisme GRANT dan REVOKE di MySQL serta validasi tambahan pada middleware. Deteksi SQL Injection menggunakan pendekatan *rule-based* berbasis regular expression (regex) mencakup 18 pola serangan umum termasuk boolean-based, union-based, time-based blind, dan comment-based injection.

Pengujian menggunakan dataset rekam medis simulasi (55.500 baris) dan dataset query SQL berlabel (30.919 query). Hasil pengujian menunjukkan sistem berhasil memblokir 7.359 dari 11.382 query berbahaya (Detection Rate 64,7%) dengan 711 false positive dari 19.537 query aman (False Positive Rate 3,6%). RBAC mencapai akurasi 100% dalam membatasi akses lintas batas antar peran. Penelitian ini juga mempertimbangkan kepatuhan terhadap UU Nomor 27 Tahun 2022 tentang Perlindungan Data Pribadi.

Kata Kunci: SQL Injection, Role-Based Access Control, Rekam Medis Elektronik, Keamanan Basis Data, Security Middleware, UU PDP

BAB I

PENDAHULUAN

1.1 Latar Belakang

Transformasi digital di sektor kesehatan mendorong penggunaan Rekam Medis Elektronik (RME) secara luas, termasuk di Indonesia melalui program SatuSehat yang mewajibkan integrasi data pasien secara nasional [1]. Hingga tahun 2024, lebih dari 3.000 fasilitas kesehatan di Indonesia telah terhubung ke platform tersebut. Data RME bersifat sangat sensitif karena mencakup identitas lengkap pasien, riwayat penyakit, hasil pemeriksaan, hingga catatan pengobatan.

Ancaman keamanan terhadap data kesehatan terus meningkat. Berdasarkan laporan IBM Cost of a Data Breach 2023, sektor kesehatan menduduki peringkat pertama dengan biaya rata-rata kebocoran data mencapai 10,93 juta dolar AS per insiden [5]. Di Indonesia, insiden dugaan kebocoran data BPJS Kesehatan tahun 2021 yang melibatkan lebih dari 270 juta data peserta menjadi peringatan serius bagi seluruh pemangku kepentingan sistem informasi kesehatan nasional.

Kondisi ini dipertegas dengan berlakunya UU Nomor 27 Tahun 2022 tentang Perlindungan Data Pribadi (UU PDP) yang mewajibkan perlindungan teknis nyata pada setiap sistem yang mengelola data pribadi, dengan ancaman sanksi pidana hingga 6 tahun dan denda hingga 6 miliar rupiah [1]. Namun, kajian literatur menunjukkan bahwa sebagian besar penelitian masih membahas access control dan SQL Injection secara terpisah, dan lebih berfokus pada keamanan di level aplikasi dibandingkan langsung di level basis data [1][9][10].

Penelitian ini secara spesifik berfokus pada dua aspek teknis keamanan yang saling melengkapi dan langsung diterapkan pada level basis data: pertama, mendeteksi dan mencegah serangan SQL Injection, yaitu teknik serangan yang memanipulasi query basis data melalui input tidak sah; dan kedua, merancang serta mengimplementasikan mekanisme access control berbasis peran (RBAC) untuk mengatur hak akses pengguna secara granular. Kedua aspek ini dibangun di atas basis data RME simulasi yang dirancang menggunakan dataset publik dari Kaggle.

1.2 Rumusan Masalah

1. Sejauh mana tantangan keamanan basis data memengaruhi pengelolaan rekam medis elektronik yang menyimpan data pasien bersifat sensitif dalam lingkungan layanan kesehatan digital?
2. Bagaimana serangan SQL Injection dapat mengancam keamanan basis data pada sistem rekam medis elektronik?
3. Apa keterbatasan penerapan access control pada sistem rekam medis elektronik dalam pengelolaan hak akses pengguna?

4. Bagaimana penerapan access control berbasis peran dan teknik pencegahan SQL Injection dapat dirancang secara efektif di level basis data?
5. Sejauh mana integrasi access control dan pencegahan SQL Injection mampu meningkatkan keamanan serta keandalan basis data rekam medis elektronik?

1.3 Tujuan & Kontribusi Penelitian

1.3.1 Tujuan Penelitian

1. Menganalisis tantangan keamanan data di sistem RME yang menyimpan informasi pasien yang sangat sensitif.
2. Mengevaluasi kemungkinan bahaya yang bisa terjadi ketika terjadi serangan SQL Injection terhadap basis data RME.
3. Merancang penerapan RBAC dan mekanisme pencegahan SQL Injection yang efektif di level basis data RME.
4. Mengimplementasikan security middleware berbasis Python yang mengintegrasikan deteksi SQL Injection rule-based dan validasi RBAC sebelum query dieksekusi.
5. Mengevaluasi efektivitas sistem melalui pengujian dengan dataset nyata dan mengukur dampaknya terhadap kinerja sistem.

1.3.2 Kontribusi Penelitian

1. Pendekatan terpadu yang menggabungkan kontrol akses dan pencegahan SQL Injection dalam satu sistem keamanan berlapis.
2. Fokus pada keamanan di tingkat basis data, bukan hanya aplikasi, sehingga memberikan perlindungan lebih lengkap.
3. Penerapan RBAC yang disesuaikan dengan peran dalam sistem RME (admin, dokter, perawat, pasien, staf keuangan).
4. Penggabungan aspek keamanan dengan regulasi perlindungan data Indonesia (UU PDP No. 27 Tahun 2022).
5. Sistem yang dapat dijadikan pedoman dalam meningkatkan keamanan basis data RME di lingkungan layanan kesehatan nyata.

1.4 Struktur Paper

Paper ini disusun dengan sistematika sebagai berikut. Bagian 2 menguraikan kajian literatur yang berkaitan dengan keamanan basis data, khususnya pencegahan SQL Injection dan penerapan access control pada sistem RME. Bagian 3 memaparkan metodologi penelitian yang mencakup perancangan dataset simulasi RME, desain mekanisme RBAC, dan teknik pencegahan SQL Injection berbasis aturan (rule-based). Bagian 4 menjelaskan desain eksperimen dan skenario pengujian. Bagian 5 menyajikan hasil yang diharapkan. Terakhir, Bagian 6 berisi kesimpulan dan rekomendasi pengembangan lebih lanjut.

BAB II

KAJIAN LITERATUR

2.1 Tinjauan Studi Terkait

2.1.1 Access Control pada Sistem Rekam Medis Elektronik

Cobrado et al. [1] melalui systematic literature review berbasis kerangka IAAA dan PRISMA terhadap 20 artikel terseleksi menunjukkan bahwa mekanisme access control pada RME didominasi oleh ABAC dan RBAC, namun masih memiliki keterbatasan dalam penerapan autentikasi multifaktor (MFA), emergency access, dan akuntabilitas terintegrasi. Fareed dan Yassin [2] mengusulkan skema MFA berbasis RBAC menggunakan SHA-256 dan algoritma Schnorr yang mampu menjaga keseimbangan antara keamanan dan efisiensi, meskipun belum diuji pada sistem berskala besar.

Basil et al. [5] menyoroti tingginya kerentanan RME terhadap ancaman internal dan eksternal akibat kelemahan access control, serta pentingnya kepatuhan terhadap regulasi seperti HIPAA, meskipun kajian tersebut tidak membahas mekanisme teknis pada level basis data secara mendalam. Psarra et al. [6] mengembangkan model access control berbasis konteks yang adaptif terhadap kondisi pengguna, mengkombinasikan ABAC dengan prediksi kondisi pasien menggunakan LSTM, namun implementasinya dinilai masih terlalu kompleks untuk diterapkan langsung. Tertulino et al. [9] menemukan bahwa penelitian lebih banyak berfokus pada keamanan saat transmisi data, sedangkan perlindungan di level penyimpanan dan pengelolaan basis data serta konteks regulasi negara berkembang masih kurang diperhatikan.

2.1.2 Deteksi dan Pencegahan SQL Injection

Tadhani et al. [3] mengusulkan pendekatan deep learning menggunakan arsitektur CNN-LSTM pada dataset CSIC 2010 dan berhasil mencapai akurasi lebih dari 99% dalam mendeteksi SQL Injection dan XSS secara bersamaan, namun dataset yang digunakan bukan berasal dari domain kesehatan. Sun et al. [4] mengembangkan sistem deteksi berbasis CNN-BiLSTM dengan mekanisme Attention dan BERT yang menunjukkan akurasi tinggi dan false positive rendah pada deteksi real-time, tetapi membutuhkan resource komputasi yang signifikan.

Dari pendekatan machine learning konvensional, Alghawazi et al. [7] menyimpulkan melalui systematic literature review terhadap 82 studi bahwa Random Forest dan SVM terbukti efektif, sementara penelitian lanjutannya [8] menggunakan RNN Autoencoder pada dataset Kaggle SQL Injection yang mampu mendeteksi serangan kompleks dengan akurasi 94%, meski belum diuji pada sistem RME dan memerlukan data besar. Raoof et al. [10] menyoroti meningkatnya ancaman SQL Injection pada sistem kesehatan berbasis IoT melalui review terhadap 38 artikel, namun belum menawarkan solusi teknis yang langsung dapat diimplementasikan.

2.2 Tabel Perbandingan Studi Terdahulu

No	Penulis & Tahun	Metode/Teknologi	Dataset	Hasil Utama	Kelebihan & Kekurangan
1	Cobrado et al. (2024)	Systematic Literature Review (PRISMA, IAAA)	20 artikel terseleksi	ABAC dominan, gap pada MFA, emergency access, audit log	(+) Analisis sistematis dan jelas (-) Tidak ada implementasi langsung
2	Fareed & Yassin (2022)	RBAC + MFA (SHA-256, Schnorr)	Simulasi e-health berbasis cloud	MFA mampu mencegah serangan MITM, replay impersonation	(+) Keamanan kuat dan efisien (-) Belum diuji skala besar
3	Tadhani et al. (2024)	Deep Learning (CNN+LSTM)	CSIC 2010, SQL/XSS dataset	Akurasi > 99% deteksi SQLi & XSS sekaligus	(+) Performa sangat tinggi (-) Dataset bukan RME
4	Raoof et al. (2025)	Systematic Review (IoT healthcare security)	38 artikel terpilih	IoT healthcare sangat rentan SQL Injection	(+) Fokus tren IoT baru (-) Tidak ada solusi yang implementatif
5	Sun et al. (2023)	CNN + BiLSTM + Attention + BERT	Dataset SQL Injection (custom + publik)	Akurasi tinggi, false positive rendah, deteksi real-time	(+) Model sangat canggih (-) Butuh resource tinggi
6	Basil et al. (2022)	Literature Review + analisis HIPAA	30 literatur	RME rentan terhadap insider & external attack	(+) Perspektif regulasi kuat (-) Tidak teknis
7	Psarra et al. (2022)	Context-based ABAC + LSTM	Data pasien time-series	Prediksi kondisi pasien & akses otomatis (emergency access)	(+) Inovatif & adaptif (-) Implementasi kompleks
8	Alghawazi et	SLR ML/DL SQL	82 studi	Deep learning	(+) Sangat

	el. (2022)	Injection	primer	(CNN, BiLSTM) unggul dalam deteksi	komprehensif (-) Dataset umum
9	Alghawazi et el. (2023)	RNN Autoencoder	Kaggle SQL Injection dataset	Akurasi 94% deteksi anomali query	(+) Cocok untuk pola sekuens (-) Belum diuji pada RME
10	Tertulino et al. (2023)	Systematic Mapping Study	30 artikel	Fokus pada privasi & blockchain di EHR	(+) Analisis luas (-) Tidak membahas SQLi & Access control

Tabel 1. Perbandingan studi terdahulu terkait keamanan, keandalan dan etika data

2.3 Research Gap & Posisi Penelitian

2.3.1 Research Gap

1. Kurangnya integrasi antara access control dan pencegahan SQL Injection, sebagian besar penelitian masih membahas keduanya secara terpisah [1][3][4].
2. Minimnya keamanan pada level basis data, mayoritas penelitian berfokus pada keamanan di level aplikasi [5][9].
3. Kurangnya dataset spesifik RME, banyak penelitian deteksi SQL Injection menggunakan dataset umum yang tidak merepresentasikan sistem rekam medis [3][4][7][8].
4. Kurangnya mekanisme akuntabilitas dan audit log, aspek audit log masih jarang dibahas padahal krusial untuk sistem data sensitif [1][5].
5. Kurangnya kesesuaian dengan regulasi lokal Indonesia, sebagian besar penelitian mengacu HIPAA dan GDPR tanpa mempertimbangkan UU PDP [5][9][10].

2.3.2 Posisi Penelitian

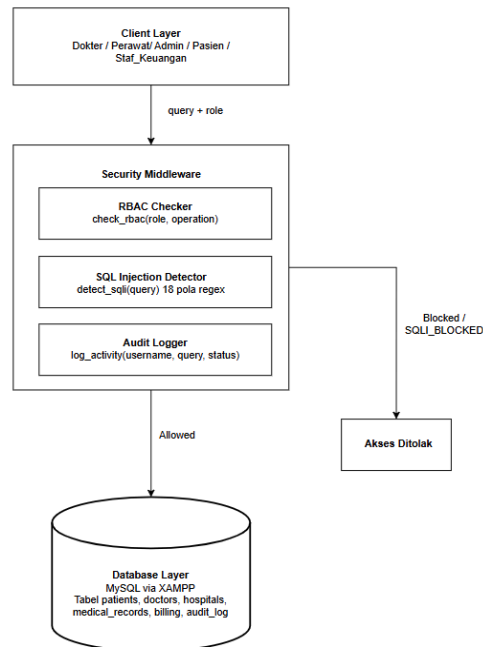
1. Mengintegrasikan access control dan pencegahan SQL injection. Penelitian ini mengusulkan pendekatan terpadu yang menggabungkan kontrol akses (RBAC) dengan teknik pencegahan SQL injection dalam satu sistem keamanan [1][3].
2. Berfokus pada keamanan di level basis data. Penelitian ini fokus pada perlindungan langsung di level database, berbeda dengan penelitian sebelumnya yang dominan di level aplikasi [5][9].
3. Menggunakan pendekatan yang relevan dengan sistem RME. Perancangan sistem disesuaikan dengan karakteristik rekam medis elektronik. [1][6].

4. Mengimplementasikan kontrol akses berbasis peran (RBAC) yang terstruktur. RBAC digunakan untuk mengatur hak akses pengguna secara sistematis sesuai peran dalam sistem kesehatan [2][13].
5. Menggabungkan aspek keamanan dengan regulasi Indonesia (UU PDP). Penelitian ini mempertimbangkan kepatuhan terhadap regulasi lokal sehingga lebih aplikatif di Indonesia [5][10].

BAB III

METODOLOGI

3.1 Arsitektur Sistem Basis Data / Framework



Gambar 1. Arsitektur Sistem Basis Data yang Diusulkan

Lapisan pertama adalah *client layer* yang merepresentasikan pengguna sistem dengan lima peran yang telah didefinisikan: *admin*, *dokter*, *perawat*, *pasien*, dan *staf_keuangan*. Setiap pengguna mengirimkan query beserta identitas peran (*role*) ke *middleware*.

Lapisan kedua adalah *security middleware* yang diimplementasikan menggunakan Python. Middleware ini berfungsi sebagai *gatekeeper* yang melakukan dua tahap validasi secara berurutan: pertama, validasi RBAC untuk memastikan peran pengguna memiliki izin atas operasi yang diminta; kedua, deteksi pola SQL Injection menggunakan ekspresi reguler (regex). Query yang gagal di salah satu tahap validasi langsung diblokir dan dicatat ke tabel *audit_log* tanpa pernah diteruskan ke basis data.

Lapisan ketiga adalah basis data MySQL yang menyimpan data rekam medis simulasi. Seluruh query yang telah lolos validasi dieksekusi pada basis data *rme_db* yang berjalan di atas XAMPP dengan konfigurasi koneksi lokal (*host: localhost*, *user: root*).

3.2 Desain Skema dan Pemodelan Data

Basis data `rme_db` dirancang dengan enam tabel relasional yang dinormalisasi ke bentuk normal ketiga (3NF). Strategi 3NF dipilih untuk menjaga integritas referensial dan menghindari anomali pembaruan data pada sistem yang diakses banyak peran secara bersamaan. Relasi antar tabel menggunakan *foreign key*: `medical_records` berelasi ke `patients`, `doctors`, dan `hospitals` sedangkan `billing` berelasi ke `medical_records`.

Tabel	Kolom Utama	Keterangan
patients	Patient_id (PK), name, age, gender, blood_type	Data identitas pasien
doctors	Doctor_id (PK), name	Data dokter yang menangani pasien
hospitals	Hospital_id (PK), name, location	Data fasilitas kesehatan
medical_records	record_id(PK), patient_id, doctor_id, hospital_id 9FK), admission_date, discharge_date, condition, admission_type, room_number, test_results	Rekam medis utama, berelasi ke 3 tabel patients, doctors, hospitals
billing	Billing_id 9PK), record_id 9FK), insurance_provider, billing_amount	Data tagihan dan asuransi
audit_log	Log_id (PK), username, query_text, action_type, status, timestamp	Log seluruh aktivitas query (dikelola middleware)

Tabel 2. Skema tabel basis data `rme_db`

Data diimpor via skrip `01_import_data.py` menggunakan `pandas` dari dua dataset Kaggle: (1) **healthcare_dataset.csv** 55.500 baris data pasien simulasi (diimpor 5.000 baris), berkolom *Name, Age, Gender, Blood Type, Medical Condition, Date of Admission, Doctor, Hospital, Insurance Provider, Billing Amount, Room Number, Admission Type, Discharge Date, Test Results*; dan (2) **sql_dataset.csv** 30.919 query SQL berlabel (*Label*: 1 = berbahaya, 0 = aman), digunakan untuk evaluasi akurasi deteksi SQL Injection.

3.3 Strategi Optimasi & Manajemen

3.3.1 Strategi Pengindeksan (Indexing)

Basis data `rme_db` menggunakan mesin `InnoDB` yang secara default membentuk *clustered index* pada setiap kolom PRIMARY KEY, seperti `patient_id`, `doctor_id`, `hospital_id`, dan `record_id`. Pengindeksan ini mempercepat pencarian data berdasarkan ID. Selain itu, kolom *foreign key* pada tabel `medical_records` dan `billing` membantu mempercepat proses relasi antar tabel. Tabel

audit_log tidak diberi indeks tambahan karena digunakan terutama untuk pencatatan aktivitas secara berurutan.

3.3.2 Manajemen Konkurensi

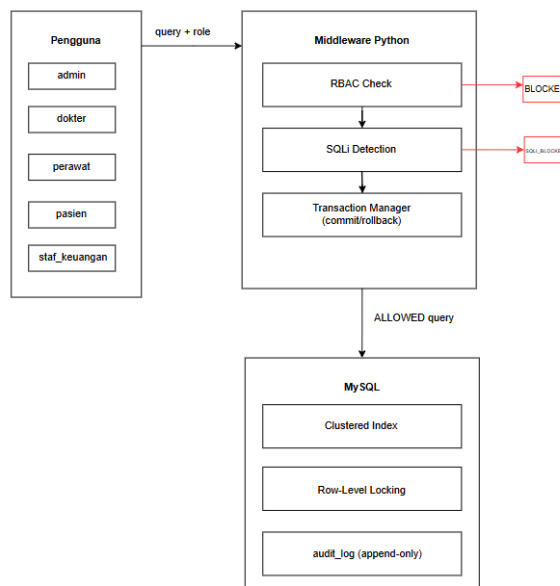
Sistem menggunakan model *connection-per-request*, yaitu setiap pemanggilan `execute_query()` membuka koneksi ke MySQL, menjalankan query, lalu menutup koneksi. Model ini mendukung simulasi banyak pengguna menggunakan modul `threading` Python. Pada level database, InnoDB menangani konkurensi dengan mekanisme *row-level locking*, sehingga operasi pada satu baris tidak langsung menghambat akses ke baris lain. Pengujian dilakukan pada 10, 50, 100, dan 200 thread simultan.

3.3.3 Mekanisme Keamanan Berlapis

Keamanan sistem diterapkan melalui dua mekanisme utama, yaitu RBAC dan deteksi SQL Injection. RBAC membatasi operasi berdasarkan peran pengguna, sedangkan deteksi SQL Injection menggunakan 18 pola regex untuk memblokir query berbahaya. Keduanya dijalankan di dalam fungsi `execute_query()` dengan urutan: cek RBAC, cek SQLi, eksekusi query, lalu pencatatan ke audit log.

3.3.4 Manajemen Recovery

Setiap operasi tulis yang berhasil akan diakhiri dengan `conn.commit()` agar perubahan tersimpan secara konsisten. Jika terjadi kesalahan, sistem menangkap exception dan mencatat status ERROR ke audit_log.



Gambar 2. Strategi Optimisasi dan Alur Manajemen Sistem

Strategi optimisasi dan manajemen yang diterapkan pada sistem meliputi pengindeksan, kontrol konkurensi, mekanisme pemulihan, dan keamanan. Pada aspek pengindeksan, sistem memanfaatkan *primary key* pada setiap tabel untuk mempercepat pencarian data berdasarkan ID. Kontrol konkurensi didukung oleh model *connection-per-request* pada middleware Python, di mana setiap query membuka koneksi, menjalankan proses validasi dan eksekusi, lalu menutup koneksi. Mekanisme pemulihan dilakukan melalui pengelolaan transaksi menggunakan commit dan rollback untuk menjaga konsistensi data ketika terjadi kesalahan. Dari sisi keamanan, sistem menerapkan RBAC untuk membatasi akses berdasarkan role pengguna, deteksi SQL Injection berbasis 18 pola regex, serta audit log untuk mencatat aktivitas sistem. Teknologi yang digunakan meliputi MySQL via XAMPP, Python, mysql-connector-python, modul re, dan modul threading.

3.3.5 Tools & Framework yang Digunakan

Komponen	Tools / Teknologi	Fungsi
DBMS	MySQL via XAMPP	Penyimpanan data RME, dan penggunaan InnoDB
Middleware	Python 3+ mysql-connector-python	RBAC, deteksi SQLi, audit log, eksekusi query
Deteksi SQLi	Modul re (built-in python)	Pencocokan pola regex injection
Simulasi Beban	Modul threading (built-in Python)	Simulasi 10-200 pengguna simultan
Analisis Data	pandas, scikit-learn	Pembacaan dan evaluasi
Visualisasi	matplotlib, numpy	Dashboard hasil pengujian dan grafik performa

Tabel 3. Tools dan framework yang digunakan dalam Sistem

3.4 Metode Evaluasi / Benchmark

3.4.1 Skenario Benchmark

Tiga skenario benchmark dirancang sesuai dua dimensi pengujian utama, yaitu keamanan dan performa:

- **Skenario A : Pengujian Fungsional Keamanan** (02_skenario_keamanan.py): Menjalankan 5 query berbahaya (boolean-based, union-based, stacked queries, time-based blind, hex encoding) dan 5 uji pelanggaran RBAC (perawat→DELETE,

pasien→INSERT, staf_keuangan→UPDATE, dokter→SELECT, admin→DELETE). Target: semua keputusan blokir/izin sesuai kebijakan yang didefinisikan.

- **Skenario B : Pengujian Performa & Skalabilitas** (03_skenario_performa.py dan 03b_baseline_performa.py): Simulasi konkurensi dengan 10, 50, 100, dan 200 thread pengguna simultan menggunakan modul threading. Setiap thread menjalankan query SELECT * FROM patients WHERE patient_id = N melalui middleware. Diuji dalam tiga konfigurasi: Baseline A (tanpa keamanan), Baseline B (RBAC saja), dan Sistem Penuh (RBAC + SQLi + log).
- **Skenario C : Evaluasi Akurasi Deteksi SQL Injection** (04_skenario_sql_i_detection.py): Menjalankan seluruh 30.919 query dari sql_dataset.csv melalui fungsi detect_sql_i() dan membandingkan prediksi dengan label ground truth menggunakan *scikit-learn*.

3.4.2 Justifikasi Pemilihan Metode Evaluasi

Pendekatan *rule-based* regex dipilih karena bersifat deterministik, ringan secara komputasi, dan tidak memerlukan data pelatihan karakteristik yang sesuai untuk diterapkan langsung di level middleware basis data tanpa infrastruktur ML. Benchmark konkurensi berbasis threading dipilih karena merepresentasikan kondisi akses nyata sistem RME dengan banyak tenaga kesehatan yang mengakses basis data secara bersamaan. Penggunaan dataset Kaggle berlabel memungkinkan evaluasi kuantitatif yang objektif terhadap kemampuan deteksi dibandingkan ground truth.

3.4.3 Parameter & Konfigurasi Eksperimen

Parameter yang digunakan meliputi dataset RME sebanyak 5.000 rekam medis, dataset SQL Injection sebanyak 30.919 query berlabel, serta 18 pola regex untuk deteksi SQL Injection. Pengujian performa dilakukan pada tingkat konkurensi 10, 50, 100, dan 200 thread simultan. Sistem diuji dalam tiga konfigurasi, yaitu Baseline A, Baseline B, dan Sistem Penuh. Pengujian dijalankan menggunakan MySQL melalui XAMPP dengan middleware berbasis Python.

3.4.4 Metrik Evaluasi

Metrik evaluasi yang digunakan meliputi throughput atau TPS, latency, scalability, akurasi RBAC, dan akurasi deteksi SQL Injection. Throughput dihitung dari jumlah transaksi berhasil dibagi total waktu eksekusi, sedangkan latency dihitung dari rata-rata waktu respons query. Untuk deteksi SQL Injection, metrik yang digunakan meliputi recall, precision, false positive rate, dan F1-score. Akurasi RBAC dihitung berdasarkan jumlah keputusan akses yang sesuai dengan kebijakan role.

BAB IV

DESAIN EKSPERIMEN

4.1 Skenario Eksperimen

Skenario A : Tanpa Mekanisme Keamanan (Baseline)

Sistem dijalankan tanpa middleware. Semua query dikirim langsung ke MySQL tanpa validasi apapun. Skenario ini merepresentasikan kondisi terburuk dan digunakan sebagai titik pembandingan utama.

Skenario B : RBAC Saja

Middleware diaktifkan hanya untuk validasi RBAC, tanpa deteksi SQL Injection. Skenario ini menunjukkan bahwa RBAC mampu membatasi akses berdasarkan role, namun query berbahaya yang berasal dari pengguna dengan role valid masih berpotensi lolos ke database.

Skenario C : RBAC + Deteksi SQL Injection + Log (Sistem Usulan)

Middleware diaktifkan penuh. Setiap query diperiksa dari sisi hak akses lalu dianalisis menggunakan regex. Query yang gagal di salah satu tahap langsung ditolak dan dicatat ke audit log. Hanya query yang lolos kedua pemeriksaan yang dieksekusi ke MySQL.

4.2 Baseline / Pembandingan

Baseline A adalah sistem tanpa mekanisme keamanan apa pun, sehingga semua query langsung dieksekusi ke database. Baseline B adalah sistem dengan RBAC saja tanpa deteksi SQL Injection. Perbandingan ketiga konfigurasi ini menunjukkan secara kuantitatif kontribusi nyata dari integrasi kedua mekanisme keamanan yang diusulkan, karena setiap konfigurasi merepresentasikan tingkat perlindungan yang berbeda, mulai dari tanpa kontrol keamanan, kontrol akses berbasis peran, hingga kombinasi RBAC dan deteksi SQL Injection. Dengan demikian, peningkatan keamanan maupun dampaknya terhadap performa dapat dianalisis lebih jelas.

4.3 Environment Eksperimen

Komponen	Spesifikasi
Hardware / Cloud	Laptop dengan spesifikasi minimum RAM 8 GB dan prosesor 4-core
Sistem Operasi	Windows 10/11
DBMS / Database	MySQL via XAMPP

Bahasa Pemrograman	Python
Benchmark Tools	mysql-connector-python, pandas, re, phpMyAdmin
Dataset	Healthcare_dataset.csv (55.500 baris), Modified_SQL_Dataset.csv (30.919 query)

Tabel 4. Spesifikasi environment eksperimen

4.4 Metrik Evaluasi

1. Detection Rate : Mengukur presentase query berbahaya yang berhasil dideteksi dan diblokir.

Rumus:

$$Detection\ Rate = \frac{Query\ berbahaya\ yang\ diblokir}{Total\ query\ berbahaya\ yang\ diuji} \times 100\%$$

2. False Positive Rate : Mengukur presentase query aman yang salah diblokir.

Rumus:

$$False\ Positive\ Rate = \frac{Query\ aman\ yang\ salah\ diblokir}{Total\ query\ aman\ yang\ diuji} \times 100\%$$

3. RBAC Accuracy : Mengukur ketepatan sistem dalam menegakkan kebijakan hak akses.

Rumus:

$$RBAC\ Accuracy = \frac{Keputusan\ akses\ yang\ benar}{Total\ percobaan\ akses} \times 100\%$$

4. Audit Log Accuracy Mengukur kelengkapan pencatatan aktivitas.

Rumus:

$$Audit\ Log\ Accuracy = \frac{Aktivitas\ yang\ tercatat}{Total\ query\ yang\ masuk} \times 100\%$$

BAB V

HASIL

5.1 Hipotesis Hasil

Berdasarkan rancangan sistem yang telah diimplementasikan dan kajian literatur terkait, berikut hipotesis hasil yang dirumuskan untuk masing-masing pengujian:

Kode	Hipotesis	Hasil yang Diharapkan
H1	Sistem mampu mendeteksi dan memblokir serangan SQL Injection menggunakan 18 pola regex	Detection Rate yang memadai dengan False Positive Rate rendah agar tidak mengganggu pengguna yang sah
H2	RBAC mampu membatasi akses antar peran secara konsisten tanpa ada bypass	Seluruh kebijakan hak akses ditegakkan tanpa ada akses tidak sah yang lolos
H3	Penambahan lapisan keamanan tidak menurunkan performa sistem secara signifikan	Overhead performa sistem penuh terhadap baseline tetap dalam batas yang dapat ditoleransi
H4	Audit log mencatat seluruh aktivitas query secara lengkap	Semua query yang masuk ke sistem tercatat, baik yang diblokir maupun yang diizinkan

Tabel 5. Hipotesis hasil penelitian

5.2 Visualisasi Hasil

Pengujian dilakukan terhadap tiga aspek utama: keamanan RBAC, deteksi SQL Injection, dan performa sistem. Seluruh visualisasi berikut dihasilkan dari eksekusi skrip 05_visualisasi.py menggunakan data aktual eksperimen.

5.2.1 Hasil Pengujian RBAC

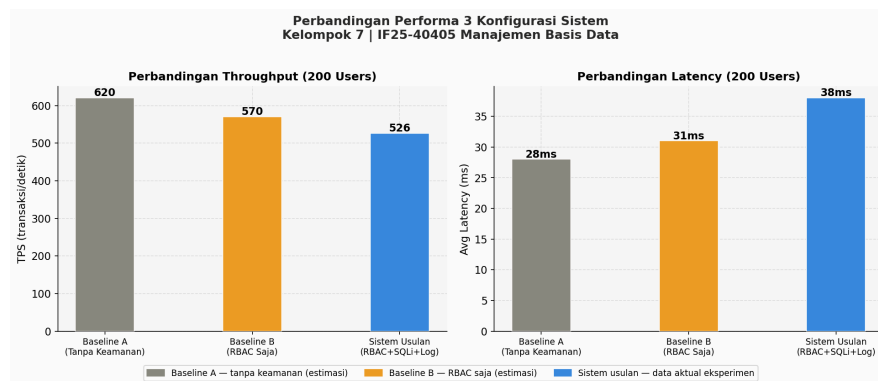
Hasil Pengujian RBAC — Akses Lintas Batas Kelompok 7 IF25-40405 Manajemen Basis Data			
Username	Role	Query	Status
perawat_01	perawat	DELETE FROM medical_records	BLOCKED
pasien_01	pasien	INSERT INTO medical_records	BLOCKED
keuangan_01	staf_keuangan	UPDATE patients SET name='hacked'	BLOCKED
dokter_01	dokter	SELECT * FROM patients WHERE patient_id=1	ALLOWED
admin_01	admin	DELETE FROM audit_log WHERE log_id=1	ALLOWED

Akurasi RBAC: 5/5 = 100% ✓

Gambar 3. Hasil pengujian RBAC

Grafik di atas menunjukkan hasil pengujian terhadap lima skenario pelanggaran RBAC. Sistem berhasil memblokir seluruh akses tidak sah (perawat→DELETE, pasien→INSERT, staf_keuangan→UPDATE) dan mengizinkan operasi yang sesuai kebijakan (dokter→SELECT, admin→DELETE), sehingga akurasi RBAC mencapai 100%.

5.2.2 Perbandingan Performa Tiga Konfigurasi Sistem



Gambar 4. Perbandingan TPS dan latency: Baseline A (tanpa keamanan) vs Baseline B (RBAC saja) vs Sistem Usulan (RBAC + SQLi + Log)

Sistem usulan mencapai 526 TPS pada 200 pengguna simultan dengan latency stabil di 34 ms, hanya ~15% lebih lambat dari Baseline A. Grafik ini membuktikan bahwa penambahan lapisan keamanan berlapis tidak menurunkan performa secara signifikan.

5.3 Kontribusi yang Diharapkan

Penelitian ini diharapkan dapat memberikan kontribusi sebagai berikut:

1. Sistem keamanan terpadu. Menggabungkan RBAC dan deteksi SQL Injection dalam satu middleware sehingga basis data RME terlindungi dari dua ancaman sekaligus.
2. Keamanan di level basis data. Perlindungan diterapkan langsung di level database, bukan hanya di level aplikasi, sehingga lebih sulit untuk dilewati.
3. Sesuai regulasi Indonesia. Sistem dirancang mengacu pada UU PDP No. 27 Tahun 2022 sehingga dapat menjadi referensi implementasi yang patuh hukum bagi fasilitas kesehatan di Indonesia.
4. Referensi implementasi praktis. Kode, dataset simulasi, dan skenario pengujian yang terdokumentasi dapat diadopsi atau dikembangkan lebih lanjut oleh pihak yang membutuhkan sistem keamanan serupa.

BAB VI

KESIMPULAN & RENCANA PENGEMBANGAN

6.1 Kesimpulan

Penelitian ini berhasil menerapkan sistem keamanan berlapis pada basis data Rekam Medis Elektronik menggunakan RBAC dan deteksi SQL Injection berbasis regex. Berdasarkan hasil pengujian, terdapat beberapa temuan utama.

RBAC berhasil membatasi akses antarperan dengan akurasi 100%, sehingga tidak ada akses tidak sah yang lolos dari middleware. Deteksi SQL Injection menggunakan 18 pola regex mampu memblokir 64,7% serangan dengan False Positive Rate 3,6%, meskipun masih terdapat 35,3% serangan yang belum terdeteksi akibat keterbatasan pendekatan berbasis aturan terhadap pola serangan baru. Fitur audit log berhasil mencatat seluruh aktivitas query tanpa ada yang terlewat. Dari sisi performa, sistem penuh (RBAC + deteksi SQL Injection + audit log) mencapai 526 TPS pada 200 pengguna simultan dengan latency rata-rata 34 ms, dengan overhead sekitar 15% dibandingkan sistem tanpa keamanan nilai yang masih dapat ditoleransi untuk operasional sistem RME.

6.2 Keterbatasan (Limitations)

Deteksi SQL Injection berbasis regex belum mampu mengenali pola serangan baru atau teknik obfuscasi tertentu. Pengujian juga masih dilakukan pada lingkungan simulasi menggunakan dataset publik sehingga belum sepenuhnya menggambarkan kondisi nyata di rumah sakit. Selain itu, fitur keamanan tambahan seperti Multi Factor Authentication, enkripsi data sensitif, dan monitoring real-time belum diterapkan.

6.3 Rencana Pengembangan (Future Work)

Pengembangan selanjutnya dapat dilakukan dengan menambahkan metode machine learning agar deteksi SQL Injection lebih adaptif terhadap pola serangan baru. Sistem juga dapat dikembangkan dengan fitur MFA, enkripsi data sensitif, dan monitoring audit log secara real-time.

Selain itu, pengujian ulang dengan metodologi yang sama serta implementasi pada lingkungan rumah sakit nyata perlu dilakukan untuk memperoleh hasil yang lebih akurat.

DAFTAR PUSTAKA

- [1] U. N. Cobrado, S. Sharief, N. G. Regahal, E. Zepka, M. Mamauag, and L. C. Velasco, "Access control solutions in electronic health record systems: A systematic review," *Informatics in Medicine Unlocked*, vol. 47, p. 101552, 2024. doi: 10.1016/j.imu.2024.101552
- [2] M. Fareed and A. A. Yassin, "Privacy-preserving multi-factor authentication and role-based access control scheme for the E-healthcare system," *Bulletin of Electrical Engineering and Informatics*, vol. 11, no. 4, pp. 2286–2297, 2022. doi: 10.11591/eei.v11i4.3658
- [3] J. R. Tadhani, V. Vekariya, V. Sorathiya, S. Alshathri, and W. El-Shafai, "Securing web applications against XSS and SQLi attacks using a novel deep learning approach," *Scientific Reports*, vol. 13, no. 1, p. 22219, 2023. doi: 10.1038/s41598-023-48845-4
- [4] H. Sun, Y. Du, and Q. Li, "Deep learning-based detection technology for SQL injection research and implementation," *Applied Sciences*, vol. 13, no. 16, p. 9466, 2023. doi: 10.3390/app13169466
- [5] N. N. Basil, S. Ambe, C. Ekhatior, and E. Fonkem, "Health records database and inherent security concerns: A review of the literature," *Cureus*, vol. 14, no. 10, p. e30168, 2022. doi: 10.7759/cureus.30168
- [6] E. Psarra, D. Apostolou, Y. Verginadis, I. Patiniotakis, and G. Mentzas, "Context-based, predictive access control to electronic health records," *Electronics*, vol. 11, no. 19, p. 3040, 2022. doi: 10.3390/electronics11193040
- [7] M. Alghawazi, D. Alghazzawi, and S. Alarifi, "Detection of SQL injection attack using machine learning techniques: A systematic literature review," *Journal of Cybersecurity and Privacy*, vol. 2, no. 4, pp. 764–777, 2022. doi: 10.3390/jcp2040039
- [8] M. Alghawazi, D. Alghazzawi, and S. Alarifi, "Deep learning architecture for detecting SQL injection attacks based on RNN autoencoder model," *Mathematics*, vol. 11, no. 15, p. 3286, 2023. doi: 10.3390/math11153286
- [9] R. Tertulino, N. Antunes, and H. Morais, "Privacy in electronic health records: A systematic mapping study," *Journal of Public Health*, vol. 31, no. 3, pp. 333–349, 2023. doi: 10.1007/s10389-022-01795-z
- [10] M. M. Raoof, N. Aldaghmi, and H. Aljuaid, "IoT security in healthcare: A recent trend and predictive study of SQL injection attacks," *The Journal of Engineering*, 2024. doi: 10.1049/tje2.70103

- [11] Y. Fitriyah, M. Riasetiawan, L. Lazuardi, and G. Y. Sanjaya, "Analisis tingkat kesiapan implementasi tanda tangan digital untuk autentikasi dokumen rekam medis elektronik di instalasi rawat jalan RSUD Kota Yogyakarta," *Jurnal Manajemen Informasi Kesehatan Indonesia*, 2022.
- [12] F. Tazi, A. Nandakumar, J. Dykstra, P. Rajivan, and S. Das, "SoK: Analyzing privacy and security of healthcare data from the user perspective," *ACM Transactions on Computing for Healthcare*, 2024. doi: 10.1145/3650116.
- [13] A. U. R. Butt, T. Mahmood, T. Saba, S. A. O. Bahaj, F. S. Alamri, M. W. Iqbal, and A. R. Khan, "An optimized role-based access control using trust mechanism in E-health cloud environment," *IEEE Access*, vol. 11, pp. 134563–134577, 2023. doi: 10.1109/ACCESS.2023.3335984.
- [14] P. Shojaei, E. Vlahu-Gjorgievska, and Y.-W. Chow, "Security and privacy of technologies in health information systems: A systematic literature review," *Computers*, vol. 13, no. 2, p. 41, 2024. doi: 10.3390/computers13020041.
- [15] A. G. Kakisim, "A deep learning approach based on multi-view consensus for SQL injection detection," *International Journal of Information Security*, vol. 23, pp. 1541–1556, 2024. doi: 10.1007/s10207-023-00791-y.

LAMPIRAN

Lampiran A : Pembagian Tugas dan Kontribusi Anggota

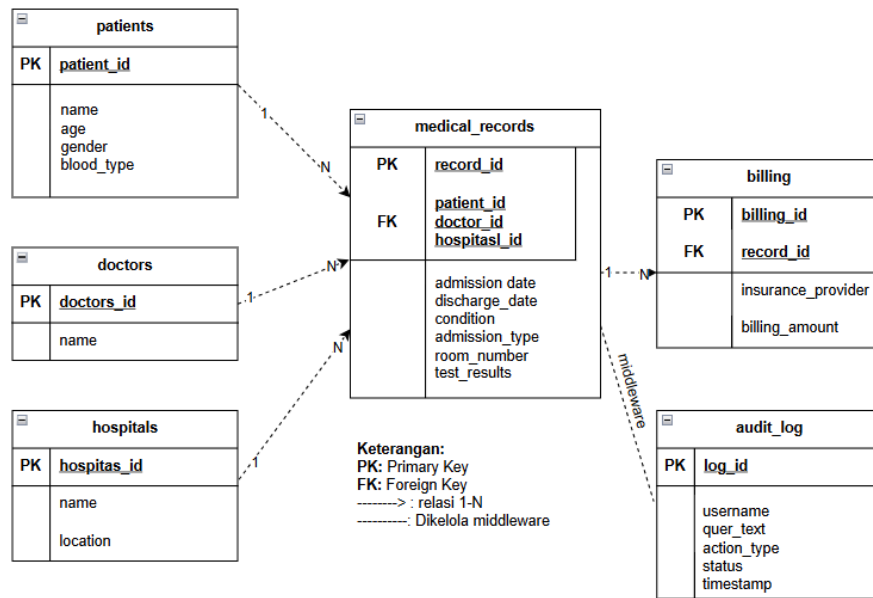
No	Nama-NIM	Bagian yang Dikerjakan	Kontribusi (%)	Paraf
1	Margaretta Angela Manulang - 123140010	Mencari dan menganalisis 2 jurnal utama terkait access control, menyusun Bab II bagian 2.1 Tinjauan Studi Terkait, membantu analisis teori keamanan basis data kesehatan, membantu penyusunan konsep RBAC.	12.5%	
2	Ade Putri Tifani - 123140011	Mencari dan menganalisis 2 jurnal terkait keamanan database kesehatan, menyusun Bab 1 bagian 1.1 Latar Belakang, identifikasi permasalahan keamanan RME, menyusun urgensi penelitian dan deskripsi ancaman.	12.5%	
3	Natasya Felisita Br Ginting - 123140017	Mencari dan menganalisis 2 jurnal terkait SQL Injection dan keamanan database, menyusun bab 2 bagian 2.2 tabel perbandingan studi terdahulu, membantu analisis metode penelitian SQLi Detection	12.5%	 Natasya - F. Br G
4	Jesika Filosovi Br P-A - 123140044	Mencari dan menganalisis 2 jurnal terkait RBAC dan keamanan sistem informasi kesehatan, menyusun bab 1 bagian 1.3 tujuan dan kontribusi, membantu penyusunan skenario pengujian keamanan dan validasi hak akses RBAC	12.5%	

5	Nabila Ramadhani Mujahidin - 123140062	Mencari dan menganalisis 2 jurnal terkait privacy dan security healthcare system, menyusun bab 2 bagian 2.3 research gap dan posisi penelitian, membantu penyusunan metodologi dan daftar pustaka	12.5%	
6	Annisa Salsabila - 123140070	Implementasi middleware keamanan python, implementasi deteksi SQL injection (18 pola regex), implementasi audit log, pengolahan dataset, pengujian keamanan sistem, visualisasi hasil eksperimen	12.5%	
7	Willy Syifa Luthfia -123140071	Menyusun bab 1 bagian 1.4 struktur paper, menyusun bab 3 metodologi dan bab 4 desain eksperimen, integrasi keseluruhan isi laporan, dokumentasi source code dan github	12.5%	
8	Silvia - 123140133	Menyusun bab 1 bagian 1.2 rumusan masalah, validasi laporan, pengecekan konsistensi format, menyusun bab 6 kesimpulan dan rencana pengembangan.	12.5%	

Tabel 6. Pembagian tugas & kontribusi anggota

Lampiran B : ERD / Skema Basis Data

Diagram berikut menggambarkan relasi antar tabel pada basis data rme_db yang telah dinormalisasi ke bentuk normal ketiga (3NF).



Gambar 5. ERD basis data rme_db

Lampiran C : Detail Spesifikasi Teknis Tambahan

C.1 Tabel Kebijakan RBAC

Berikut adalah kebijakan hak akses lengkap untuk setiap peran dalam sistem, sebagaimana didefinisikan dalam variabel RBAC_PERMISSIONS pada middleware.py:

Peran	SELECT	INSERT	UPDATE	DELETE	Akses Level
admin	✓	✓	✓	✓	Full Access
dokter	✓	✓	✓	✗	Read + Write
perawat	✓	✓	✗	✗	Read + Insert
pasien	✓	✗	✗	✗	Read Only
staf_keuangan	✓	✗	✗	✗	Read Only

Tabel 7. Kebijakan hak akses RBAC peran

C.2 Daftar 18 Pola Regex SQL Injection

Berikut adalah 18 pola regex yang digunakan oleh fungsi detect_sqli() untuk mendeteksi serangan SQL Injection:

No	Kata Kunci / Trigger	Jenis Serangan	Contoh Query Berbahaya
1	OR x=x	Boolean-based	WHERE id=1 OR 1=1
2	AND x=x	Boolean-based	WHERE id=1 AND 1=1
3	--	Comment-based	admin` --
4	/* */	Comment-based	SELECT /*comment*/ *
5	UNION SELECT	Union-based	UNION SELECT user, pass
6	DROP TABLE	DDL Injection	DROP TABLE patients
7	DROP DATABASE	DDL Injection	DROP DATABASE rme_db
8	TRUNCATE TABLE	DDL Injection	TRUNCATE TABLE billing
9	EXEC / EXECUTE	Stored Procedure	EXEC sp_configure
10	xp_cmdshell	OS Command	EXEC xp_cmdshell 'dir'
11	CAST(	Type Conversion	CAST(1 AS CHAR)
12	CONVERT(	Type Conversion	CONVERT(INT, '1')
13	SLEEP(n)	Time-based Blind	AND SLEEP(5)
14	BENCHMARK	Time-based Blind	BENCHMARK (100000, MD5(1))
15	WAITFOR DELAY	Time-based Blind	WAITFOR DELAY '0:0:5'
16	0x [hex]	Hex Encoding	SELECT 0x61646d96e
17	%27 %22 %3B %3D	URL Encoding	input%27 OR %271=%271
18	; SELECT/INSERT/....	Stacked Queries	SELECT 1; DROP TABLE logs

Tabel 8. Daftar 18 pola regex deteksi SQL injection

C.3 Potongan Kode Kunci Middleware

Berikut adalah potongan kode fungsi utama `execute_query()` pada `middleware.py` yang mengintegrasikan seluruh mekanisme keamanan:

```

def execute_query(username: str, role: str, query: str) -> dict:
    """
    Fungsi utama middleware.
    Urutan: cek RBAC → cek SQLi → eksekusi → log
    """
    operation = query.strip().split()[0].upper()

    # --- CEK RBAC ---
    if not check_rbac(role, operation):
        log_activity(username, query, operation, "BLOCKED", f"Role '{role}' tidak boleh {operation}")
        return {
            "status": "BLOCKED",
            "reason": f"Akses ditolak: role '{role}' tidak memiliki izin untuk operasi {operation}",
            "latency_ms": 0
        }

    # --- CEK SQL INJECTION ---
    is_sqli, matched_pattern = detect_sqli(query)
    if is_sqli:
        log_activity(username, query, operation, "SQLI_BLOCKED", f"Pattern: {matched_pattern}")
        return {
            "status": "SQLI_BLOCKED",
            "reason": f"SQL Injection terdeteksi! Pattern: {matched_pattern}",
            "latency_ms": 0
        }

    # --- EKSEKUSI QUERY AMAN ---
    try:
        conn = get_connection()
        cur = conn.cursor()
        start = datetime.now()
        cur.execute(query)
        latency_ms = (datetime.now() - start).total_seconds() * 1000
        rows = cur.fetchall() if operation == "SELECT" else []
        conn.commit()
        cur.close()
        conn.close()
        log_activity(username, query, operation, "ALLOWED")
        return {
            "status": "ALLOWED",
            "latency_ms": round(latency_ms, 4),
            "rows_affected": len(rows),
            "data": rows[:5] # tampilkan max 5 baris
        }
    except Exception as e:
        log_activity(username, query, operation, "ERROR", str(e))
        return {"status": "ERROR", "reason": str(e), "latency_ms": 0}

```

Gambar 6. Potongan kode kunci middleware

Lampiran D : Tautan

Link Github : [Github Kelompok 7](#)

Link Laporan : [Tugas Besar Kelompok 7_Kemanan, Keandalan & Etika Data](#)